NBA Project

Config

```
# === GCP PROJECT/WIF CONFIG ===
export PROJECT_ID="levizion-ai"
export PROJECT_NUMBER="20311219227"
export REGION="us-central1"

# Artifact Registry
export AR_REPO="us-central1-docker.pkg.dev/${PROJECT_ID}/levizion"

# Buckets
export DATA_BUCKET="gs://${PROJECT_ID}-oob-data"
export MLFLOW_BUCKET="gs://${PROJECT_ID}-mlflow"

# Service Accounts
export SA_DEPLOYER="github-deployer@${PROJECT_ID}.iam.gserviceaccount.com"
export SA_RUN_INFER="run-infer@${PROJECT_ID}.iam.gserviceaccount.com"
export SA_JOBS_RUNNER="jobs-runner@${PROJECT_ID}.iam.gserviceaccount.com"
export SA_BATCH_RUNNER="batch-runner@${PROJECT_ID}.iam.gserviceaccount.com"

# WIF provider
export GCP_WIF_PROVIDER="projects/${PROJECT_NUMBER}/locations/global/workloadIdentityPools/github-pool/providers/github-provider"

# GitHub repo identity
export GH_OWNER="ry4ntr1"
```

```
export GH_REPO_NAME="levizion"
export GITHUB_REPO="${GH_OWNER}/${GH_REPO_NAME}"

# gcloud defaults
gcloud config set project "$PROJECT_ID"
gcloud config set run/region "$REGION"
gcloud config set artifacts/location "$REGION"
gcloud config set compute/region "$REGION"
gcloud config set compute/zone "${REGION}-a"

# Deployed services (live)
API_UI_URL="https://api-ui-tzd5dsvqcq-uc.a.run.app"      # Cloud Run (CPU)
INFER_GPU_URL="https://infer-gpu-tzd5dsvqcq-uc.a.run.app" # Cloud Run (G
PU L4)
```

Current Progress

▼ GCP NBA

Here's a single, well-structured handoff document you can drop into your repo (e.g., `PROJECT_HANDBOOK.md`). It captures the north star, architecture, decisions, data schemas, evaluation, cloud setup, CI/CD, and a machine-readable **StateCard** so another chat session (or teammate) can pick up exactly where you left off.

Basketball OOB MVP — Project Handbook (Handoff-Ready)

> Purpose: This document is the source of truth for the Automatic Out-of-Bounds (OOB) MVP and related features. Keep it updated. Paste this file (plus the StateCard at the top)

> into any new chat along with your current repo status, and the assistant can continue the work seamlessly.

---

0) StateCard (fill in and keep current)

```
# === STATECARD v1 ===
project:
  name: bball-oob-mvp
  repo_url: <https://github.com/OWNER/REPO>
  default_branch: main
  last_commit_sha: <SHA>
  last_updated_utc: <YYYY-MM-DD HH:MM>

cloud:
  provider: gcp
  project_id: <GCP_PROJECT_ID>
  region: us-central1
  artifact_registry: us-central1-docker.pkg.dev/<GCP_PROJECT_ID>/bball
  buckets:
    data: gs://<GCP_PROJECT_ID>-oob-data
    mlflow: gs://<GCP_PROJECT_ID>-mlflow
  services:
    api_ui_url: <https://api-ui-xxxxxxxx.a.run.app>       # Cloud Run (CPU)
    infer_gpu_url: <https://infer-gpu-xxxxxxxx.a.run.app>  # Cloud Run (GPU L4)
    mlflow_url: <https://mlflow-xxxxxxxx.a.run.app>       # optional
  identities:
    github_wif_provider: projects/<num>/locations/global/workloadIdentityPools/github-pool/providers/github-provider
    deployer_sa: github-deployer@<GCP_PROJECT_ID>.iam.gserviceaccount.com

data:
  detection_dataset:
```

```
    source: Roboflow "basketball-players-fy4c2"
    format: YOLOv8
    path_or_uri: <local path or gs://...>
    classes: [player, ref, ball, hoop, scoreboard]
  labeled_event_clips:
    count: <N>           # short clips (3–8s)
    location: gs://<...>/events/
  calibration:
    method: manual_4pt_per_clip
    cache_dir: gs://<...>/calibration/
    canonical_court: NBA_94×50

models:
  detector:
    family: ultralytics-yolo
    weights_uri: gs://<...>/models/detector.pt
    onnx_uri: gs://<...>/models/detector.onnx
    imgsz: 1280
    notes: <fine-tune ball later>
  tracker:
    players_refs: bytetrack_or_botsort
    ball: kalman_2d
  pose_near_boundary:
    enabled: false  # true once added
  jersey:
    mode: color_clustering_k2
  export:
    format: onnx  # engine optional later

pipeline_status:
  ingestion: ready|wip|blocked
  training: ready|wip|blocked
  inference_service: ready|wip|blocked
  ui: ready|wip|blocked
  oob_fsm: ready|wip|blocked
  homography: ready|wip|blocked
```

```
  possession: ready|wip|blocked
  team_id: ready|wip|blocked
  shot_pass: ready|wip|blocked
  eval_harness: ready|wip|blocked
  ci_cd: ready|wip|blocked

metrics_benchmark:
  dataset_summary: "<clips, arenas, fps, res>"
  detection_map50: <float>
  tracking_idf1: <float>
  oob_f1_overall: <float>
  oob_f1_live: <float>
  possession_acc: <float>
  team_id_acc: <float>
  shot_f1: <float>
  pass_f1: <float>
  throughput_fps_720p: <float>

open_issues:
  - id: O1
    title: "<short title>"
    detail: "<what's needed>"
    owner: me
    status: todo|doing|done
```

1) North-Star MVP

Demo: Automatic Out-of-Bounds with context, near real-time on wide broadcast footage.

Also (time-boxed): team classification, ball possession, shot & pass detection, court mapping (homography), small web UI (clip playback, overlays, timeline), and basic observability.

Acceptance (portfolio-grade):

- OOB F1 ≥ **0.80** overall; ≥ **0.70** on *live play* subtype

- Possession accuracy ≥ **0.85**
- Team ID ≥ **0.90** on half-court possessions
- Throughput ≥ **25–30 FPS** @ 720p on a single GPU
- One-command demo via Docker/Cloud Run

Non-goals (MVP): multi-camera fusion, whistle audio, number OCR at scale, 24/7 serving (we scale to zero).

---

2) Final Decisions (so far)

- **Cloud**: **Google Cloud** for seamless, low-idle cost:
  - **Cloud Run (GPU L4)** for inference (scale-to-zero)
  - **Cloud Run Jobs** for ingestion/transcode (CPU)
  - **Cloud Batch (GPU)** for training (ephemeral VMs)
  - **Artifact Registry** for images; **GCS** for datasets & artifacts
- **Detector**: Ultralytics YOLO (Roboflow set; ball fine-tuned)
- **Tracking**: ByteTrack/BoT-SORT (players/refs), Kalman (ball)
- **Homography**: manual **4-point per clip** to canonical NBA 94×50 court
- **Team ID**: Torso color K=2 clustering (EMA stabilized); refs handled separately
- **Possession**: nearest-neighbor (ball↔player) + velocity & persistence gates
- **OOB FSM**: `LIVE` / `AFTER_SCORE` / `INBOUNDING`; strict feet (pose near boundary in v2)
- **Eval**: 30–60 short clips benchmark across 5–10 games; honest metrics + 2–3 ablations
- **CI/CD**: GitHub Actions with **Workload Identity Federation** (keyless) → build/push/deploy to Cloud Run/Batch
- **Budget**: under $200; everything scales to zero when idle

---

3) Architecture Overview

```
flowchart LR
  subgraph Ingestion (on-demand)
    U[User Upload / Raw GCS] --> JR[Cloud Run Job (FFmpeg/OpenCV)]
    JR --> GCSproc[(GCS: processed clips, frames, labels)]
  end

  subgraph Training (on-demand)
    GCSproc --> B[Cloud Batch Job (GPU L4)]
    B --> GCSmodels[(GCS: models, metrics, MLflow artifacts)]
  end

  subgraph Inference & UI (scale-to-zero)
    UI[Cloud Run: API+UI (CPU)] -->|/infer| INF[Cloud Run: Inference (GPU L
4)]
    INF -->|events| UI
    GCSmodels --> INF
    GCSproc --> UI
  end

  subgraph Observability
    UI --> MET[Cloud Run / Metrics endpoint]
    INF --> MET
  end
```

4) Repo Layout (canonical)

```
data/                 # DVC-tracked pointers; NOT large binaries in git
models/                 # local weights during dev; final in GCS
src/bball_oob/            # geometry, jersey, possession, utils
services/
  api/                # FastAPI endpoints: /infer, /events, /metrics
  infer/              # GPU inference server (Triton/ONNXRuntime optional)
```

```
  detector.py            # YOLO wrapper
  tracker.py             # ByteTrack/BoT-SORT hooks, Ball Kalman
  event_engine/oob_fsm.py # state machine
pipeline/
  ingest.py              # frame/clip ops; used by Cloud Run Job
  train_detector.py      # training script (logs to MLflow)
  eval.py                # metrics/ablations
tools/
  calibrate_four_point.py # per-clip H estimation
ui/
  app.py                 # Streamlit demo
deploy/
  dockerfiles/           # Dockerfile.api, Dockerfile.ui
  cloudrun/              # (optional) service YAMLs
infra/
  terraform/             # buckets, artifact registry, service accounts, etc.
.github/workflows/       # CI/CD (build, deploy, jobs, train)
tests/                   # unit tests (geom, FSM, jersey)
configs/default.yaml     # central config (thresholds, paths)
PROJECT_HANDBOOK.md      # this file
```

---

5) Data & Schemas

Storage: `gs://<project>-oob-data` (raw, processed, events, calibration, models)

Labeling (CVAT):

- Target: **200–300** short clips (3–8s) labeled for:
  - `shot`, `pass`, `possession` intervals
  - `oob` with subtype: `{live, inbounding, after_score}`

Formats

- `tracks.parquet`
  - Columns: `frame_id:int, track_id:int, cls:str, x1:float, y1:float, x2:float, y2:float, conf:float`
- `events.jsonl`

- Each line:

```
{
  "clip_id":"<id>",
  "type":"OOB|SHOT|PASS|POSSESSION",
  "subtype":"live|inbounding|after_score|null",
  "start_frame":123,
  "end_frame":145,
  "confidence":0.0-1.0,
  "meta": { "player_id": 42, "team": 0 }
}
```

- `calibration/<clip_id>_H.npy` — 3×3 homography matrix (numpy save)
- `court/canonical_nba.png` — diagram only for visual overlays

DVC: Track *pointers* to data, not binaries in git. Remotes → GCS.

---

6) Models & Algorithms (MVP)

6.1 Detection

- **Ultralytics YOLO**; classes: player, ref, ball, hoop (scoreboard as optional)
- Train on Roboflow export (YOLO format). Fine-tune **ball** if weak.
- Export to **ONNX** for inference; optional TensorRT later.

6.2 Tracking

- **Players/Refs**: ByteTrack or BoT-SORT (A/B choose better IDF1)
- **Ball**: lightweight **Kalman**; update when ball detection exists, predict otherwise

6.3 Court Mapping (Homography)

- Per-clip **4-point** calibration → canonical NBA 94×50
- Project feet (bbox bottom corners; later ankles/toes from pose) into court space

6.4 Team Classification

- Torso crop → **Lab/HSV** → **K=2** clustering for first ~300 frames
- Lock cluster centers; per track → cosine distance + EMA smoothing
- Refs remain separate class (never assigned to teams)

6.5 Possession

- Possessor = nearest player to ball with distance threshold + stationary gate
- Flip possession only if **modal team over last N frames** changes; suppress during high ball speed (airborne)

6.6 Events

OOB (hero)

- `LIVE`: if any projected **foot polygon vertex** is outside court for ≥ **K frames** → `OOB_TURNOVER`
- Guard-rails:
  - `AFTER_SCORE`: entered when ball intersects hoop ROI; brief dead ball
  - `INBOUNDING`: when ball OOB & stationary near baseline ≥0.7s; exit on pass to inbounds
- (v2) **Pose-near-boundary**: run pose only when a foot bbox is within X px of boundary, then use ankle/toe keypoints

Shot

- Ball vertical velocity sign flip + parabolic arc + hoop proximity; optional net ROI motion spike

Pass

- Ball speed spike + direction change + possessor flip within 0.4–0.8s; distance > threshold

(Stretch) Set classification

- 4–6 coarse buckets from 4-s trajectories; start rule-based, then small GRU/1D-CNN

---

7) State Machine (pseudo-Python)

```
class GameState(Enum):
    LIVE=0; AFTER_SCORE=1; INBOUNDING=2

state = GameState.LIVE
t_since_score = 0.0
t_ball_stationary_oob = 0.0

def update(frame, dt=1/30):
    global state, t_since_score, t_ball_stationary_oob
    ball_in_hoop = hoop_contact(ball, hoop)            # implement
    ball_oob = is_oob(ball.xy, court_poly)             # project + polygon test
    ball_stationary = ball.speed < v0
    near_baseline = dist_to_baseline(ball.xy) < d0

    if ball_in_hoop:
        state = GameState.AFTER_SCORE
        t_since_score = 0.0
        t_ball_stationary_oob = 0.0

    if state == GameState.AFTER_SCORE:
        t_since_score += dt
        if ball_oob and ball_stationary and near_baseline:
            t_ball_stationary_oob += dt
            if t_ball_stationary_oob > 0.7:
                state = GameState.INBOUNDING
        elif t_since_score > 4.0:
            state = GameState.LIVE

    elif state == GameState.INBOUNDING:
        if pass_from_oob_to_inbounds(ball_traj):
            state = GameState.LIVE

    elif state == GameState.LIVE:
```

```
        for p in players:
            if foot_oob(p.foot_pts, court_poly, k_frames=3):
                emit_event("OOB_TURNOVER", player=p)
```

---

8) Evaluation & Ablations

Benchmark: 30–60 short clips (3–8 s), 5–10 games, variety of levels.

Metrics:

- Detection: mAP50 / mAP50-95
- Tracking: IDF1, MOTA
- OOB: Precision/Recall/F1 (overall + `{live, inbounding, after_score}`)
- Possession: framewise accuracy; switch latency histogram
- Shot/Pass: F1

Ablations (report deltas):

- Tracker choice (ByteTrack vs BoT-SORT)
- Homography quality (4-pt vs 8+ anchors)
- Jersey method (color vs small torso CNN)

Error taxonomy (CSV):

`clip_id, frame, event_type, pred, gt, error_tag`

Tags: homography drift, occlusion, jersey confusion, ball lost, pose miss, state mis-switch, etc.

---

9) Observability & QA

- **/metrics** endpoint (Prometheus format) with:
  - fps, stage latency p50/p95 (detector/tracker/event), event counts
- **Structured logs** (JSON) tagged with clip/frame ids
- **Misfire sampler**: persist top N false positives/negatives to `data/mistakes/` for relabel

- **Unit tests**: geometry (homography/project/containment), FSM transitions, jersey clustering lock
- **CI**: run `pre-commit`, `pytest`, build docker images, push on `main`

---

10) Cloud Setup (GCP) — Summary

- **Artifact Registry**: `bball` (Docker)
- **GCS buckets**: `<proj>-oob-data`, `<proj>-mlflow`
- **Cloud Run (CPU)**: `api-ui` (min 0)
- **Cloud Run (GPU L4)**: `infer-gpu` (min 0, concurrency 1)
- **Cloud Run Jobs (CPU)**: `ingest` (args: `-input=gs://...`)
- **Cloud Batch (GPU L4)**: `train-<run_id>` jobs for training
- **WIF** GitHub → GCP: keyless CI/CD

> All services scale to zero when idle; Batch/Jobs are ephemeral.

---

11) CI/CD Workflows (GitHub Actions)

- `ci.yml`: install, test, build images (no deploy)
- `deploy-api-ui.yml`: build & deploy **api-ui** to Cloud Run (CPU)
- `deploy-infer-gpu.yml`: build & deploy **infer-gpu** to Cloud Run (GPU L4)
- `ingest-run.yml`: run **Cloud Run Job** with `input_uri`
- `train-batch.yml`: submit **Cloud Batch** training job
- `infra-apply.yml` (optional): Terraform plan/apply

Secrets/vars needed:

`GCP_PROJECT_ID, GCP_REGION, GCP_WIF_PROVIDER, GCP_SERVICE_ACCOUNT_EMAIL, AR_REPO`

---

12) Performance Plan

- **Target**: 25–30 FPS @ 720p (single L4)
- **Toggles**:
  - Resize frames to max width 1280
  - Detector FP16; small/medium backbone
  - Run **pose only near boundary**
  - Batch frames in micro-batches (2–4) if using Triton/ORRT
  - Pin OpenCV decode threads

---

13) Cost Controls

- Cloud Run services: `min-instances: 0`, **scale-to-zero**
- Cloud Batch: VMs created/destroyed per training job
- GCS Standard tier; delete intermediates weekly
- Keep egress low: export short, compressed replays for sharing

---

14) Backlog (ordered)

1. **Detector training** (baseline) on Roboflow export; copy `best.pt` to GCS
2. **Tracking A/B** (ByteTrack vs BoT-SORT) on 2–3 games; pick winner
3. **4-pt homography** tool into pipeline; overlay court; store `H.npy` per clip
4. **Team color clustering K=2**; per-track smoothing; visualize
5. **Possession** heuristic (dist + velocity + hysteresis)
6. **OOB FSM** wired; bbox-foot first; log events
7. **UI overlays + timeline**; “Export replay” mp4
8. **Label 200–300 clips**; build eval harness; compute baseline metrics
9. **Pose-near-boundary** (switch on if latency OK)
10. **Shots & passes** heuristics; add to eval
11. **Ablations & README polish** (tables + figure)

15) Decision Log (ADR snapshots)

ID	Date	Decision	Context	Status
ADR-001	YYYY-MM-DD	Choose GCP Cloud Run GPU + Batch	Scale-to-zero + easy CI/CD	Accepted
ADR-002	YYYY-MM-DD	Manual 4-point homography	Generalize across arenas quickly	Accepted
ADR-003	YYYY-MM-DD	Jersey via K=2 color clustering	Fast v1; upgrade path to CNN	Accepted
ADR-004	YYYY-MM-DD	Pose only near boundary	Strict OOB without latency hit	Planned

(Extend as you decide things.)

16) API Surface (MVP)

- `GET /health` → `{ok:true}`
- `GET /metrics` → Prometheus metrics
- `POST /infer` → `{detections, tracks, events}` for a frame or buffer (consider base64 or GCS URI)
- `GET /events?clip_id=...` → returns events with timestamps and types

17) How to Work This Project (for future you / another assistant)

1. **Read the StateCard** at the top. If fields are missing, ask for them or infer safely.
2. **Sync context**: check repo URLs, commit SHA, service URLs.

3. **Operate through CI/CD**: prefer Action workflows (`ingest-run`, `train-batch`, `deploy-*`) over manual cloud clicks.
4. **Update metrics_benchmark** after each major run; push changes.
5. **Log new decisions** in the ADR table; add TODOs to `open_issues`.
6. **Keep costs low**: shut down long-lived jobs, rely on scale-to-zero, compress outputs.

If starting mid-stream, the quickest way to resume is:

- Run `deploy-api-ui.yml` and `deploy-infer-gpu.yml` (GPU endpoint)
- Execute `ingest-run.yml` on one raw clip (provide `gs://` path)
- Confirm homography + overlays; then wire/verify OOB FSM
- Submit `train-batch.yml` only if the detector needs re-training

---

18) Demo Script (5 minutes)

1. Open UI (Cloud Run) → load sample clip from GCS
2. Toggle overlays (players/refs/ball/foot points/court)
3. Play timeline: show OOB turnover vs inbounding vs after-score
4. Click “Export replay” → download annotated mp4
5. Show `/metrics` briefly (FPS, latencies)
6. Flash the README table: metrics + ablation result

---

19) Open Questions (track here)

- Do we include inbounder stepping on the line in v1?
- Is scoreboard OCR needed for your footage mix?
- What exact latency requirement (ms/frame) should the README claim?

---

20) Quickstart Commands (local)

```
# Dev
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pre-commit install
uvicorn services.api.main:app --reload
streamlit run ui/app.py

# Train (Ultralytics)
yolo task=detect mode=train data=data/detector/data.yaml model=yolov8
s.pt imgsz=1280 epochs=50 device=0

# Track
yolo task=detect mode=track model=models/detector.pt source=path/to/cl
ip.mp4 tracker=bytetrack.yaml conf=0.25 save=True

# Calibrate (4 points)
python tools/calibrate_four_point.py --image frame.jpg --out data/calibratio
n/<clip>_H.npy
```

---

21) Glossary

- **OOB**: Out-of-bounds
- **Homography (H)**: 3×3 transform mapping image pixels → canonical court coordinates
- **K-frames**: number of consecutive frames required to confirm an OOB event
- **EMA**: exponential moving average (smoothing over time)
- **Scale-to-zero**: service has **min instances = 0**, incurring no compute cost when idle

---

Appendix A — Court Boundary (outer polygon only)

- Canonical NBA: 94 × 50 (arbitrary pixel map: 940 × 500)
- Outer polygon = rectangle; add arcs later only if needed for visuals

Appendix B — Event Output (example)

```
{
  "clip_id": "gameA_q2_03",
  "frame": 1624,
  "ts": 54.13,
  "type": "OOB_TURNOVER",
  "state": "LIVE",
  "player_id": 12,
  "team": 1,
  "court_xy": [3.2, -24.8],
  "confidence": 0.92
}
```

Final note

Drop this file at the repo root and **keep the StateCard updated**. When you start a new chat, paste:

1. This `PROJECT_HANDBOOK.md`
2. The **current StateCard** (filled)
3. Any logs/metrics or errors you want the assistant to act on

That's all a capable assistant needs to continue your build, re-run CI/CD, debug metrics, or ship the next feature without missing a beat.

▼ Basketball Detection & Tracking Project Checklist

Phase 1: Project Setup & Environment Preparation

☐ **1.1 Define Project Scope**

☐ List all objects to detect (players, ball, referee, hoop, jersey numbers)

☐ Define required outputs (bounding boxes, tracking IDs, team classification, statistics)

- [ ] Specify performance requirements (real-time vs post-processing, accuracy targets)
- [ ] Determine input sources (live stream, recorded video, resolution requirements)

- [ ] **1.2 Set Up Development Environment**
  - [ ] Create Python virtual environment (Python 3.8+)
  - [ ] Install CUDA and cuDNN for GPU acceleration
  - [ ] Install core dependencies:

```
pip install ultralytics opencv-python supervision numpypip install tor
ch torchvision torchaudio --index-url https://download.pytorch.org/
whl/cu118pip install lap motmetrics  # For tracking algorithmspip inst
all roboflow  # If using Roboflow datasets
```

  - [ ] Verify GPU is accessible: `python -c "import torch; print(torch.cuda.is_available())"`

Phase 2: Data Collection & Preparation

- [ ] **2.1 Gather Training Data**
  - [ ] Collect basketball game footage from various angles and lighting conditions
  - [ ] Ensure diversity: different courts, team uniforms, camera angles
  - [ ] Include edge cases: occlusions, fast movements, ball out of frame
  - [ ] Minimum dataset size: 1000+ images for robust performance

- [ ] **2.2 Data Annotation**
  - [ ] Choose annotation tool (Roboflow, LabelImg, CVAT)
  - [ ] Select dataset sources
  - [ ] Define annotation classes:
    - Players (separate by team if needed)
    - Ball

- Referee
- Hoop/Basket
- Jersey numbers (if doing OCR)

☐ Pseudo labeling

☐ Create annotation guidelines for consistency

☐ Annotate dataset with bounding boxes

☐ Split data: 70% train, 20% validation, 10% test

☐ **2.3 Data Augmentation**

☐ Apply augmentations to increase dataset robustness:

- Brightness/contrast adjustments
- Motion blur (for fast movements)
- Rotation (small angles)
- Scaling variations
- Horizontal flipping (maintain court orientation)

Phase 3: Model Selection & Training

☐ **3.1 Choose Base Models**

☐ Select YOLO version (YOLOv8 recommended for balance of speed/accuracy)

☐ Decide on model size:

- YOLOv8n (nano) - Fastest, lowest accuracy
- YOLOv8s (small) - Good balance
- YOLOv8m (medium) - Better accuracy
- YOLOv8l (large) - High accuracy, slower
- YOLOv8x (extra large) - Highest accuracy, slowest

☐ **3.2 Train Detection Models**

- [ ] Option A: Single unified model
  - [ ] Train one model to detect all classes
  - [ ] Pros: Simpler pipeline, faster inference
  - [ ] Cons: May struggle with class imbalance
- [ ] Option B: Multiple specialized models
  - [ ] Train separate models for:
    - Player detection (high accuracy needed)
    - Ball detection (small object, fast movement)
    - Jersey number detection (on cropped player regions)
  - [ ] Pros: Better per-class performance
  - [ ] Cons: More complex pipeline, slower

- [ ] **3.3 Training Configuration**
  - [ ] Set training parameters:

    epochs: 100-300batch_size: 16-32 (depending on GPU memory)img sz: 640-1280 (larger for ball detection)patience: 20-50learning_rate: 0.01

  - [ ] Monitor training metrics (mAP, loss curves)
  - [ ] Implement early stopping to prevent overfitting
  - [ ] Save best model weights

Phase 4: Tracking Implementation

- [ ] **4.1 Select Tracking Algorithm**
  - [ ] ByteTrack (Recommended for basketball):
    - Best for crowded scenes
    - Handles occlusions well
    - Good balance of speed/accuracy

- [ ] DeepSORT:
  - Uses appearance features
  - Better for similar-looking objects
  - Slightly slower
- [ ] BoT-SORT:
  - State-of-the-art accuracy
  - More computationally intensive

- [ ] **4.2 Configure Tracking Parameters**
  - [ ] Set tracking thresholds:

```
track_thresh: 0.25-0.5  # Detection confidence thresholdtrack_buffe
r: 30-60     # Frames to keep lost tracksmatch_thresh: 0.7-0.9   # Ma
tching threshold
```

  - [ ] Tune for basketball-specific motion patterns
  - [ ] Account for fast direction changes
- [ ] **4.3 Implement Track Management**
  - [ ] Handle track initialization (new players entering frame)
  - [ ] Manage track updates (continuous tracking)
  - [ ] Implement track termination (players leaving frame)
  - [ ] Add re-identification for players returning to frame

Phase 5: Multi-Model Integration

- [ ] **5.1 Design Pipeline Architecture**
  - [ ] Create modular design with clear interfaces
  - [ ] Implement parallel or sequential processing based on needs
  - [ ] Design data flow between models
- [ ] **5.2 Implement Model Fusion**

- [ ] Create unified output format
- [ ] Implement Non-Maximum Suppression (NMS) across models
- [ ] Handle overlapping detections from different models
- [ ] Merge tracking information

- [ ] **5.3 Optimize Performance**
  - [ ] Implement frame skipping for non-critical models
  - [ ] Use model quantization if needed
  - [ ] Batch process where possible
  - [ ] Profile and identify bottlenecks

Phase 6: Basketball-Specific Features

- [ ] **6.1 Team Classification**
  - [ ] Implement color-based team detection
  - [ ] Use jersey color clustering
  - [ ] Handle referee distinction

- [ ] **6.2 Ball Possession Detection**
  - [ ] Calculate distance between players and ball
  - [ ] Implement possession logic (proximity + time)
  - [ ] Handle contested possessions

- [ ] **6.3 Event Detection**
  - [ ] Shot detection (ball trajectory analysis)
  - [ ] Pass detection (ball movement between players)
  - [ ] Out-of-bounds detection

- [ ] **6.4 Court Mapping**
  - [ ] Implement homography for court projection
  - [ ] Map player positions to 2D court diagram

- [ ] Calculate player movement statistics

Phase 7: Testing & Validation

- [ ] **7.1 Unit Testing**
  - [ ] Test each model independently
  - [ ] Verify tracking consistency
  - [ ] Test edge cases (occlusions, fast movements)
- [ ] **7.2 Integration Testing**
  - [ ] Test full pipeline end-to-end
  - [ ] Verify multi-model coordination
  - [ ] Test on various video qualities
- [ ] **7.3 Performance Testing**
  - [ ] Measure FPS on target hardware
  - [ ] Profile memory usage
  - [ ] Test with long videos (memory leaks)
- [ ] **7.4 Accuracy Evaluation**
  - [ ] Calculate mAP for each class
  - [ ] Evaluate tracking metrics (MOTA, MOTP, ID switches)
  - [ ] Compare against ground truth annotations

Phase 8: Production Deployment

- [ ] **8.1 Create Application Interface**
  - [ ] Build CLI for batch processing
  - [ ] Create GUI for interactive use (optional)
  - [ ] Implement API endpoints if needed
- [ ] **8.2 Error Handling**

- [ ] Handle missing/corrupted frames
- [ ] Implement graceful degradation
- [ ] Add comprehensive logging

- [ ] **8.3 Output Generation**
  - [ ] Save annotated videos
  - [ ] Export tracking data (JSON/CSV)
  - [ ] Generate statistics reports
  - [ ] Create visualization dashboards

Phase 9: Optimization & Scaling

- [ ] **9.1 Performance Optimization**
  - [ ] Implement GPU batching
  - [ ] Use TensorRT for inference optimization
  - [ ] Consider model pruning/quantization
- [ ] **9.2 Scaling Considerations**
  - [ ] Design for parallel processing of multiple games
  - [ ] Implement cloud deployment if needed
  - [ ] Add queue system for batch processing

Phase 10: Maintenance & Improvement

- [ ] **10.1 Monitoring**
  - [ ] Track model performance over time
  - [ ] Monitor for drift in new game footage
  - [ ] Collect failure cases
- [ ] **10.2 Continuous Improvement**
  - [ ] Retrain models with new data

- [ ] Update tracking parameters based on performance
- [ ] Add new features based on user feedback

Key Decision Points

1. Single vs Multiple Models

Single Model Approach:

- ✅ Simpler implementation
- ✅ Faster inference
- ❌ May struggle with class imbalance
- ❌ Harder to optimize per-class

Multiple Model Approach:

- ✅ Better per-class optimization
- ✅ Can use specialized architectures
- ❌ More complex pipeline
- ❌ Potentially slower

2. Real-time vs Post-processing

Real-time Processing:

- Requires optimization (model size, batching)
- May need to sacrifice some accuracy
- Consider frame skipping strategies

Post-processing:

- Can use larger, more accurate models
- Allows for multiple passes
- Better for detailed analytics

3. Tracking Algorithm Selection

For Basketball, prioritize:

1. Handling of occlusions (players clustering)
2. Fast motion tracking
3. Consistent ID maintenance
4. Team-aware tracking

Success Metrics

- [ ] Detection mAP > 0.85 for players
- [ ] Detection mAP > 0.80 for ball
- [ ] Tracking MOTA > 0.70
- [ ] Processing speed > 25 FPS (for real-time)
- [ ] ID switch rate < 5%
- [ ] Ball possession accuracy > 90%

Common Pitfalls to Avoid

1. **Insufficient training data diversity** - Include various lighting conditions and camera angles
2. **Ignoring class imbalance** - Ball appears less frequently than players
3. **Over-optimizing for accuracy** - Balance with speed requirements
4. **Poor track management** - Lost tracks during occlusions
5. **Not handling edge cases** - Ball out of frame, player substitutions
6. **Inadequate testing** - Test on full-length games, not just clips

Resources & References

- YOLOv8 Documentation: https://docs.ultralytics.com/
- ByteTrack Paper: https://arxiv.org/abs/2110.06864
- Supervision Library: https://supervision.roboflow.com/

- Basketball Analytics Papers: Search for "SportVU basketball tracking"